Bubble Sort

Resumo completo — baseado em `Aula6_AnaliseOrdenacao.pdf` e `Lista6_AnaliseOrdenacao.pdf` (Exercício 4; partes dos Exercícios 1 e 7).

Propriedade	Valor
Estratégia	compara elementos **adjacentes** e troca quando estão fora de ordem
Melhor caso	$\Theta(n^2)$ na versão sem parada; $\Theta(n)$ na versão otimizada
Caso médio	$\Theta(n^2)$
Pior caso	$\Theta(n^2)$
Espaço auxiliar	$\Theta(1)$
In-place	Sim
Estável	Sim (troca só com `>`)

---

1. Ideia

Percorre o vetor comparando vizinhos `v[j]` e `v[j+1]`; se estiverem fora de ordem, troca. Ao final de cada **passagem**, o maior elemento da parte não ordenada "borbulha" até a última posição dessa parte.

Passo a passo em `[5, 2, 4, 1]`

Passagem 1 (parte não ordenada: posições 0–3):

Comparação	Ação	Vetor
5 e 2	troca	`[2, 5, 4, 1]`
5 e 4	troca	`[2, 4, 5, 1]`
5 e 1	troca	`[2, 4, 1, 5]`

O maior (5) está na posição final.

Passagem 2 (posições 0–2):

Comparação	Ação	Vetor
2 e 4	não troca	`[2, 4, 1, 5]`
4 e 1	troca	`[2, 1, 4, 5]`

Passagem 3 (posições 0–1):

Comparação	Ação	Vetor
2 e 1	troca	`[1, 2, 4, 5]`

Vetor ordenado: `[1, 2, 4, 5]`. Total: 3 + 2 + 1 = 6 comparações e 5 trocas (o vetor tinha 5 inversões).

---

2. Implementação

```
def bubble_sort(v):
    n = len(v)
    for i in range(n - 1):                # laço externo: controla as passagens
        for j in range(n - 1 - i):        # laço interno: compara vizinhos
            if v[j] > v[j + 1]:           # > (e não >=) preserva a estabilidade
                v[j], v[j + 1] = v[j + 1], v[j]
    return v
```

- Cada troca corrige uma **inversão local** (um par vizinho fora de ordem).
- O limite do laço interno **diminui** (`n - 1 - i`), pois após a passagem `i` os `i + 1` maiores já estão nas posições finais.
- **Por que `>` e não `>=`?** Com `>=`, elementos iguais seriam trocados e a ordem relativa entre eles se perderia. Não trocar iguais garante a **estabilidade**.

Versão otimizada (parada antecipada)

```
def bubble_sort_otimizado(v):
    n = len(v)
    for i in range(n - 1):
        trocou = False
        for j in range(n - 1 - i):
            if v[j] > v[j + 1]:
                v[j], v[j + 1] = v[j + 1], v[j]
                trocou = True
        if not trocou:                    # passagem sem trocas: já está ordenado
            break
    return v
```

> É preciso analisar o **código concreto**, não só o nome do algoritmo: a parada antecipada muda o melhor caso de $\Theta(n^2)$ para $\Theta(n)$.

---

3. Análise de complexidade passo a passo

3.1 Número de comparações (versão sem parada)

A passagem i (com $i = 0, 1, \ldots, n-2$) faz $n - 1 - i$ comparações:

Passagem i	0	1	2	...	$n-2$
Comparações	$n-1$	$n-2$	$n-3$	...	1

$$C(n) = \sum_{i=0}^{n-2}(n-1-i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}$$

(soma de Gauss — apêndice de 01 — Recursividade). Esse número **não depende da entrada**: mesmo um vetor ordenado faz $n(n-1)/2$ comparações.

3.2 Número de trocas

Cada troca de vizinhos corrige exatamente **uma inversão** (veja 02 — Fundamentos). Logo, trocas = inversões da entrada:

Caso	Entrada	Trocas
Melhor	já ordenada	0
Médio	permutação aleatória	$\frac{n(n-1)}{4}$
Pior	ordem inversa	$\frac{n(n-1)}{2}$

3.3 Tempo total

Com custo c_1 por comparação e c_2 por troca:

$$T(n) = c_1\frac{n(n-1)}{2} + c_2 \cdot (\text{trocas}), \qquad 0 \le \text{trocas} \le \frac{n(n-1)}{2}$$

Como o primeiro termo já é $\Theta(n^2)$ em **qualquer** entrada:

- **Versão sem parada:** melhor = médio = pior = $\Theta(n^2)$.

3.4 Versão otimizada

- **Melhor caso (vetor ordenado):** a 1ª passagem faz $n-1$ comparações, nenhuma troca, e o algoritmo para: $T(n) = c(n-1) = \Theta(n)$.
- **Pior caso (ordem inversa):** toda passagem troca; executa as $n-1$ passagens: $\Theta(n^2)$.
- **Caso médio:** ainda há $\approx n(n-1)/4$ trocas → $\Theta(n^2)$.

3.5 Espaço

Só variáveis auxiliares (`i`, `j`, `trocou`, temporário da troca): $\Theta(1)$ → **in-place**.

---

4. Exercícios resolvidos

Exercício 4 — `v = [42, 17, 8, 33, 25, 11, 49, 6, 28, 14]` **($n = 10$)**

a) Vetor após cada comparação da 1ª passagem:

Comp.	Par comparado	Ação	Vetor
1	42 e 17	troca	`[17, 42, 8, 33, 25, 11, 49, 6, 28, 14]`
2	42 e 8	troca	`[17, 8, 42, 33, 25, 11, 49, 6, 28, 14]`
3	42 e 33	troca	`[17, 8, 33, 42, 25, 11, 49, 6, 28, 14]`
4	42 e 25	troca	`[17, 8, 33, 25, 42, 11, 49, 6, 28, 14]`
5	42 e 11	troca	`[17, 8, 33, 25, 11, 42, 49, 6, 28, 14]`
6	42 e 49	não troca	`[17, 8, 33, 25, 11, 42, 49, 6, 28, 14]`
7	49 e 6	troca	`[17, 8, 33, 25, 11, 42, 6, 49, 28, 14]`
8	49 e 28	troca	`[17, 8, 33, 25, 11, 42, 6, 28, 49, 14]`
9	49 e 14	troca	`[17, 8, 33, 25, 11, 42, 6, 28, 14, 49]`

b) 1ª passagem: **9 comparações e 8 trocas**. O 49 (maior) chegou à posição final.

c) Vetor ao final de cada passagem:

Passagem	Comparações	Trocas	Vetor
1	9	8	`[17, 8, 33, 25, 11, 42, 6, 28, 14, 49]`
2	8	6	`[8, 17, 25, 11, 33, 6, 28, 14, 42, 49]`
3	7	4	`[8, 17, 11, 25, 6, 28, 14, 33, 42, 49]`
4	6	3	`[8, 11, 17, 6, 25, 14, 28, 33, 42, 49]`
5	5	2	`[8, 11, 6, 17, 14, 25, 28, 33, 42, 49]`
6	4	2	`[8, 6, 11, 14, 17, 25, 28, 33, 42, 49]`
7	3	1	`[6, 8, 11, 14, 17, 25, 28, 33, 42, 49]`
8	2	0	`[6, 8, 11, 14, 17, 25, 28, 33, 42, 49]`
9	1	0	`[6, 8, 11, 14, 17, 25, 28, 33, 42, 49]`

Total de trocas: $8 + 6 + 4 + 3 + 2 + 2 + 1 = 26$ = número de inversões do vetor.

d) Total de comparações sem parada antecipada:

$$9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 = \frac{10 \cdot 9}{2} = 45$$

e) Nessa implementação (sem parada): melhor, médio e pior caso $\Theta(n^2)$ — as comparações são sempre $n(n-1)/2$.

f) Parada antecipada: se uma passagem não faz trocas, o vetor já está ordenado e o algoritmo para. No melhor caso (vetor ordenado) basta 1 passagem com $n-1$ comparações: $\Theta(n)$. Neste vetor, a passagem 8 não tem trocas, então a versão otimizada para ali: $9+8+\cdots+2=44$ comparações (economiza só a passagem 9). O pior e o médio continuam $\Theta(n^2)$.

Exercício 1 (parte Bubble) — `[7, 3, 5, 2]`

1ª passagem: compara (7,3) troca, (7,5) troca, (7,2) troca → `[3, 5, 2, 7]`. Resolução completa e comparação com os outros algoritmos em 02 — Fundamentos.

Exercício 7 (parte Bubble)

Vetor	Sem parada	Com parada
`v1 = [16, 9, 28, 3, 21, 7, 35, 12, 5, 24]`	45 comparações, 22 trocas	44 comparações (8 passagens), 22 trocas
`v2 = [3, 5, 7, 9, 12, 16, 21, 24, 28, 35]` (ordenado)	45 comparações, 0 trocas	**9 comparações** (1 passagem), 0 trocas

Comparação com Selection e Insertion em 02 — Fundamentos.

Matérias correlatas

> **Soma de Gauss** — o total de comparações $\sum(n-1-i)=n(n-1)/2$ é a mesma soma usada na recorrência $T(n)=T(n-1)+n$. Veja 01 — Recursividade (seção 4.2 e apêndice).

> **Inversões e estabilidade** — trocas do Bubble = inversões = deslocamentos do Insertion; o uso de `>` garante estabilidade. Veja 02 — Fundamentos e 05 — Insertion Sort.

> **Selection Sort** — mesmo número de comparações, mas no máximo $n-1$ trocas. Veja 04 — Selection Sort.